

Resenha – Design by Contract

"Design by Contract" é um capítulo acadêmico escrito por Bertrand Meyer, da Interactive Software Engineering Inc., publicado como parte de um livro sobre avanços em engenharia de software orientada a objetos. O objetivo do texto é apresentar uma abordagem de desenvolvimento de software baseada em uma metáfora simples mas poderosa: a construção de software como uma sucessão de decisões contratuais documentadas. O capítulo é técnico e denso, mas escrito de forma relativamente acessível graças ao uso intensivo de analogias com contratos do mundo real e exemplos de código na linguagem Eiffel.

O ponto de partida de Meyer é uma pergunta direta: como construir software confiável? O autor argumenta que boa parte da literatura sobre programação orientada a objetos ignora completamente as questões de correção e robustez, como se a flexibilidade estrutural do paradigma fosse suficiente por si só. A resposta proposta é o Design by Contract, que parte da ideia de que qualquer chamada de rotina em um programa é análoga a um contrato entre duas partes: o cliente, que chama a rotina, e o fornecedor, que a implementa. O cliente tem obrigações (satisfazer a pré-condição) e benefícios (receber o resultado prometido). O fornecedor, por sua vez, tem a obrigação de entregar o resultado e o benefício de não precisar lidar com casos em que a pré-condição não foi cumprida.

Na parte técnica, Meyer apresenta os três mecanismos centrais da abordagem. As pré-condições, expressas com a cláusula `require`, definem o que o cliente precisa garantir antes de chamar uma rotina. As pós-condições, com `ensure`, definem o que o fornecedor garante ao término da execução. E os invariantes de classe garantem que todos os objetos daquela classe se mantenham em um estado consistente ao longo de todo o ciclo de vida. O autor usa como exemplo principal uma classe `TABLE` com uma operação de inserção `put`, detalhando como cada elemento do contrato se aplica na prática. O operador `old`, que permite referenciar o valor de uma variável antes da execução da rotina dentro de uma pós-condição, é um detalhe especialmente elegante da linguagem Eiffel que ilustra bem o nível de expressividade disponível.

Um dos pontos mais interessantes do capítulo é a crítica ao que Meyer chama de "programação defensiva", que consiste em proteger cada módulo com o máximo de verificações possível, mesmo quando essas verificações são redundantes com as já feitas pelo cliente. O autor argumenta que essa prática, embora bem-intencionada, leva ao oposto do que promete: programas mais complexos, cheios de verificações redundantes, onde é difícil encontrar o código que realmente faz algo útil. O contrato resolve isso ao deixar claro quem é responsável por cada condição de consistência, eliminando a necessidade de verificações duplas.

A seção sobre tratamento de exceções é uma das mais ricas e apresenta três leis fundamentais que todo mecanismo de exceção deveria respeitar. Meyer usa exemplos de Ada e CLU para mostrar como esses mecanismos podem ser mal utilizados, especialmente quando permitem que uma rotina falhe silenciosamente, retornando ao

chamador como se nada tivesse acontecido. A alternativa proposta em Eiffel, com as cláusulas `rescue` e `retry`, é mais restrita e disciplinada: uma rotina que não consegue cumprir seu contrato precisa ou tentar novamente com outra estratégia, ou declarar sua falha ao chamador de forma explícita.

Por fim, o capítulo aborda como o Design by Contract se aplica à herança. A chamada Regra de Redefinição estabelece que ao redefinir uma rotina em uma subclasse, a pré-condição só pode ser enfraquecida e a pós-condição só pode ser fortalecida. Isso é análogo ao comportamento esperado de um subcontratado honesto, que pode aceitar mais casos e entregar resultados melhores do que o contratante original, mas nunca o contrário.

O ponto forte do capítulo é a clareza conceitual: Meyer consegue apresentar ideias formais sem tornar a leitura árida, graças ao uso constante da metáfora do contrato e de exemplos práticos. A principal limitação é que os exemplos de código dependem inteiramente da linguagem Eiffel, o que pode distanciar leitores acostumados com Java, Python ou C++. Ainda assim, o texto é uma referência essencial para quem quer entender como especificar de forma rigorosa o comportamento de componentes de software, e os princípios apresentados são completamente transferíveis para qualquer linguagem ou paradigma.